Granular ball local consensus for graph-free node classification under label noise

Silei Cao[a], Kun Wang[a], Jingyu Zhu[b], Tianxing Wang[c,*], Bing Huang[a]

[a] *School of Computer Science, Nanjing Audit University, Nanjing 211815, PR China*
[b] *School of Statistics and Data Science, Nanjing Audit University, Nanjing 211815, PR China*
[c] *School of Business, Nanjing Audit University, Nanjing 211815, PR China*
* Corresponding author. E-mail addresses: mp2509053@stu.nau.edu.cn (Silei Cao), kun_wang@nau.edu.cn (Kun Wang), jingyuzhu77@gmail.com (Jingyu Zhu), wangtianxing@nau.edu.cn (Tianxing Wang), huangbing@nau.edu.cn (Bing Huang).

Abstract

Graph-free node classification enables prediction from node attributes when graph relations are unavailable or costly to access at deployment. However, existing noise-robust graph methods generally retain graph-dependent inference, while graph-free methods have largely been studied with clean training labels. To address this gap, we propose Granular Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification. GB-LCSS learns node representations through label-free graph contrastive learning and recursively groups nodes in the frozen representation space into granular balls. For each training node, the labels of the other training nodes in its ball provide a local consensus. A weight determined by the agreement among these labels combines this consensus with the node's observed label to form a soft target for a multilayer perceptron (MLP). This design excludes observed labels from region construction and uses shared local evidence to reduce the influence of isolated label errors. Across five datasets and five noise settings, GB-LCSS achieves or ties for the highest five-seed mean accuracy in 24 of 25 settings among seven noisy-label baselines evaluated using the same deployed MLP architecture, while the deployed classifier requires only node attributes at inference.

Keywords: label noise, node classification, graph-free inference, graph contrastive learning, granular ball computing, soft labels

1 Introduction

Node classification is a fundamental task in graph learning, where node attributes and relationships jointly support prediction. Graph neural networks (GNNs) combine these sources of information by aggregating representations from neighboring nodes [1, 2]. Their dependence on graph structure, however, creates practical difficulties at deployment. Predictions require access to the relevant neighborhoods, while retrieving and aggregating multi-hop neighbor information adds computational cost. In applications with missing relations, privacy restrictions, or low-latency requirements, this access may be unavailable or impractical. These constraints have motivated graph-free node classification methods such as Graph-MLP [3] and GLNN [4], which exploit graph structure during training but use only node attributes at inference.

Subsequent work has explored richer forms of graph-to-MLP knowledge transfer. SimMLP aligned GNN and MLP representations without supervision [5], while FF-G2M transferred low- and high-frequency information from a GNN teacher [6]. These developments broaden the ways in which graph structure can guide MLP training.

Noisy training labels pose an additional problem. Labels obtained through manual annotation, automated rules, or multiple sources may contain errors, and deep models can eventually fit

corrupted annotations [7]. On graphs, message passing may propagate the influence of mislabeled nodes to their neighbors [8]. Many existing noise-robust graph methods use topology to augment supervision [9], regulate training with noisy labels [10], or select reliable samples [11]. These methods address label noise but still require graph access at inference. Our aim is to use graph structure to improve noisy-label supervision during training while allowing the classifier to predict from node attributes alone.

One way to address this challenge is to combine an observed label with evidence from nearby training nodes. When nearby training nodes exhibit consistent class evidence, aggregating their labels can reduce the influence of isolated annotation errors. Neighborhood-based methods for noisy-label learning have exploited related forms of local consistency [12, 13]. For graph data, a graph contrastive representation learned from node attributes and connectivity without labels provides a basis for identifying such local groups [14]. These groups can then supply label evidence without allowing noisy annotations to determine their membership.

Granular ball computing provides a regional representation for this purpose. It groups samples into coarse-grained units summarized by centers and radii [15]. Subsequent studies have investigated granular ball sampling for noisy or imbalanced classification [16] and refined adaptive ball generation [17]. Depending on the task, ball construction can be guided by label purity or geometric criteria [18]. Geometry-based construction is suitable here because it allows local groups to be formed before the training labels are aggregated. Each group can then provide shared label statistics for constructing supervision for its members.

Building on this idea, we propose Granular Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification with noisy labels and graph-free inference. As shown in Fig. 1, GB-LCSS first learns node representations through label-free graph contrastive learning. It then recursively applies 2-means to node representations in the frozen representation space, assigning each node to exactly one granular ball. For each training node, the labels of the other training nodes in its ball provide a local consensus. A weight determined by the agreement among these labels combines this consensus with the node's observed label to form a soft target. These targets are used to train an MLP that receives only node attributes at inference.

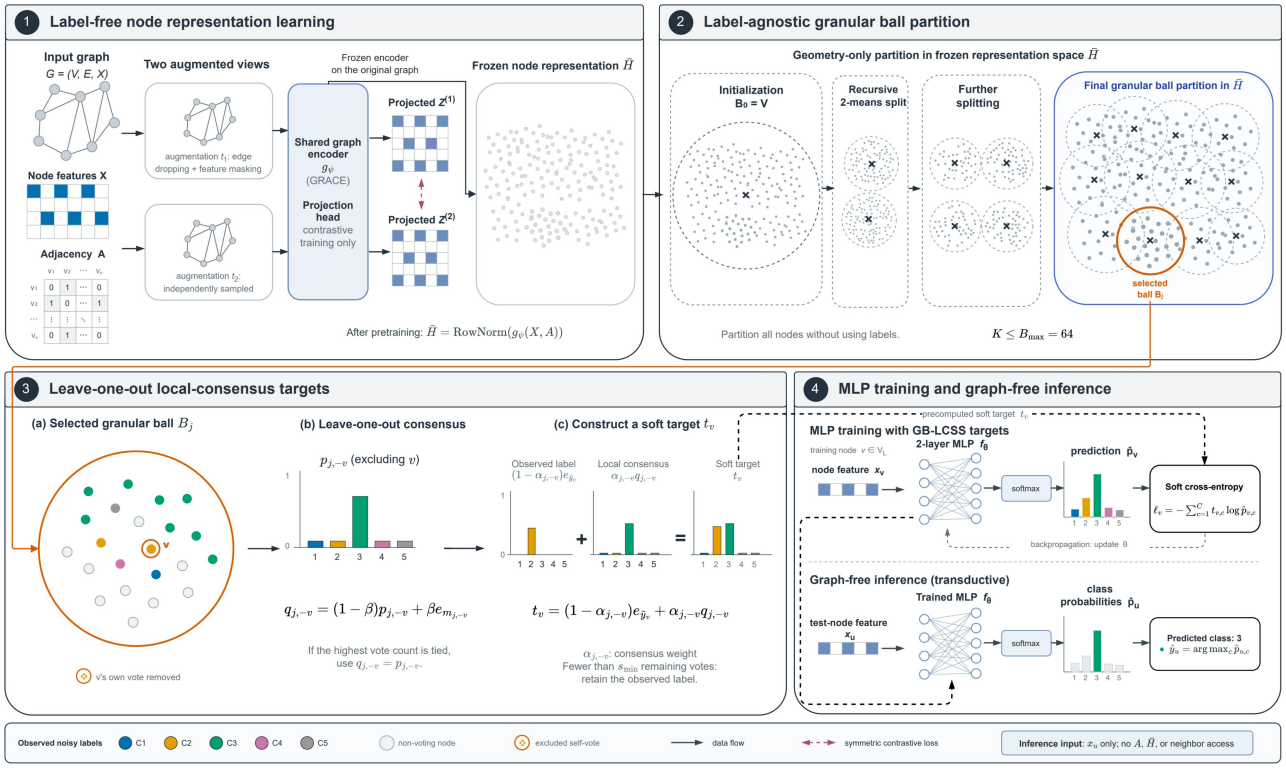


Fig. 1. Overview of GB-LCSS. Stage 1 learns a label-free representation, and Stage 2 constructs granular balls in that representation space. Stage 3 enlarges a representative ball to show how a local-consensus target is formed. Stage 4 trains the deployed MLP with the resulting soft targets. Hollow nodes do not vote. Ball outlines are schematic, and each node belongs to exactly one ball.

Experiments on five datasets and five noise settings show that GB-LCSS attains or ties for the highest five-seed mean accuracy in 24 of 25 settings among seven noisy-label baselines evaluated using the same deployed MLP architecture. Controlled comparisons examine how the labels of training nodes within each region contribute to target construction, and a graph-teacher extension evaluates the same targets for distillation.

The main contributions are summarized as follows:

(1) We organize nodes into granular balls in a frozen, label-free graph representation space, assigning each node to one local group. This separates region construction from noisy-label aggregation.

(2) We develop consensus-weighted soft supervision within these regions. Each training node combines its observed label with the labels of the other training nodes in its ball, with their agreement determining the contribution of local consensus. We analyze the reliability of local voting under independent symmetric label noise and derive an exact decomposition of soft-target error.

(3) We evaluate GB-LCSS on five datasets and five noise settings against noisy-label and graph-assisted baselines. Controlled studies examine regional membership, target construction, and representation learning, and a graph-teacher extension evaluates the same targets for distillation.

The remainder of this paper is organized as follows. Section 2 reviews related work, and Section 3 presents GB-LCSS and its graph-teacher extension. Section 4 describes the experimental setup, comparative results, ablations, extensions, and discussion. Section 5 concludes the paper.

2 Related work

We review learning with noisy labels and graph-free node classification, followed by graph contrastive representation learning and granular ball computing.

2.1 Learning with noisy labels

Methods for learning with noisy labels aim to reduce the influence of corrupted annotations through sample selection, target refinement, or robust optimization. Sample selection exploits the tendency of deep networks to learn clean patterns before memorizing noise. Co-teaching [19], for example, exchanged small-loss samples between two networks, while JoCoR [20] also encouraged prediction agreement. DivideMix [21] treated clean and noisy samples within a semi-supervised framework. Beyond loss-based selection, FINE [22] assessed class-wise feature directions, and CT [23] tracked prediction confidence. Knockoffs-SPR [24] used penalized regression and knockoff filters to select clean samples while controlling false selections under its theoretical assumptions. Label agreement among nearest neighbors in a feature space has also been used directly: Deep k-NN [13] filtered samples whose labels disagree with those of their neighbors, and neighbor-consistency regularization [12] encouraged each prediction to match those of its neighbors.

Other methods address label noise by modifying the training loss. SCE [25] combined cross-entropy with reverse cross-entropy, while ELR [26] regularized training to limit noise memorization. ANL [27] used active negative loss functions, and LogitClip [28] constrained logit norms. These strategies address annotation noise without explicitly modeling graph relations between samples.

2.2 Noise-robust learning on graphs

Graph structure can provide additional evidence for assessing label reliability, while message passing may also spread the influence of supervision errors [8]. UnionNET [29] used neighborhood-based label estimates for reweighting and correction, NRGNN [9] augmented supervision through structural information and pseudo-labeling, and RTGNN [10] coordinated two networks to regulate noisy supervision.

Label reliability can also be assessed through graph topology and model behavior. TSS [11] used topological sample selection, while TFR [30] reconstructed topological features. ProCon [31] used peer-informed label consistency, and GD² [32] identified unreliable supervision through prediction discrepancies between two views. GNN-CFGD [33] combined coarse- and fine-grained sample division to address label noise and sparsity.

Beyond assessing label reliability, several methods have constructed additional supervision. DND-NET [34] used neighborhood uncertainty to reweight pseudo-label supervision, whereas RNCGLN [35] combined graph contrastive learning with pseudo graphs and pseudo labels to address graph and label noise. Prototype-Guided Supervision [36] formed class-level prototypes by aggregating representations of nodes with the same observed label. BO-NNC [37] and SAMTD [38] improved supervision through multiple graph teachers.

Other methods address label noise by modifying graph structure, neighborhood aggregation, or the training objective. GTA [39] adapted graph topology, MILAB [40] combined information bottleneck-guided aggregation with adversarial consistency, and HALF [41] fused low- and high-pass losses according to local homophily to identify reliable labels. NoisyGL [8] established a unified evaluation framework for learning with noisy labels on graphs.

2.3 Graph-free node classification

Methods for graph-free node classification exploit graph information during training while avoiding online message passing at inference. Graph-MLP [3] incorporated adjacency through a neighborhood-contrastive objective, whereas GLNN [4] distilled predictions from a GNN teacher into an MLP. Both methods deploy an MLP that receives only node attributes. GSDN [42] used neighborhood self-distillation, SimMLP [5] aligned GNN and MLP representations without supervision, and ES-MLP [43] incorporated edge splitting into contrastive training. Tian et al. [44] analyzed the effectiveness, robustness, and efficiency of graph-trained MLPs, while SA-MLP [45] investigated structure-aware distillation. These methods differ in how graph information guides training and whether precomputed structural features are retained at inference.

Subsequent distillation methods differ in the teacher information they transfer. FF-G2M [6] separated low- and high-frequency knowledge, KRD [46] assessed knowledge reliability through the stability of prediction entropy under noise perturbations, and VQGraph [47] encoded local structures as discrete tokens. PGKD [48] used class prototypes to guide structural knowledge distillation from GNNs to MLPs, whereas TINED [49] combined teacher parameter injection with Dirichlet-energy distillation. AdaGMLP [50], ProGMLP [51], and SALE-MLP [52] investigated adaptive transfer, accuracy–cost trade-offs, and structure-aware latent representations, respectively. Yu et al. [53] theoretically analyzed how label alignment in distillation allows an MLP to learn a GNN teacher's implicit graph bias.

A related line of work uses graph structure to construct soft training targets. PosteL [54] constructed posterior soft labels from neighborhood labels and graph-wide label statistics. ALS [55] used neighborhood label distributions and adaptively updated smoothed targets during graph training, while SALS [56] incorporated structural information into the target label distribution. These methods draw on graph neighborhoods and label statistics to construct their targets. GB-LCSS also modifies the training targets, but defines regions by proximity in a frozen, label-free representation and excludes each node's own label from the consensus used to adjust its target.

2.4 Graph contrastive representation learning

Self-supervised graph learning derives supervision from node attributes and graph connectivity rather than annotated class labels [57]. Graph contrastive learning constructs different views of a graph by perturbing its structure or attributes. A common node-level objective brings representations of the same node across views closer together while distinguishing them from those of other nodes. The resulting node vectors provide a basis for comparing nodes without using their observed class labels.

GRACE [14] implements this approach through random edge dropping and feature masking, with a shared graph encoder processing the two views. GB-LCSS adopts this node-level contrastive objective to learn the representations used for granular ball construction. After pretraining, the encoder is frozen and nodes are grouped according to their representations. The observed training labels are used only afterwards to construct soft supervision. This separates the information used to form local groups from the potentially noisy labels aggregated within them.

2.5 Granular ball computing

Granular ball computing represents samples using coarse-grained hyperspherical regions, which GBkNN used to classify queries [15]. Granular ball sampling extended this representation to sample selection for noisy or imbalanced classification [16]. MGNR [58] used granular ball representations to construct multi-granularity neighborhood relationships for classification and clustering. For noisy-label learning, MgCNL [59] used multi-granularity balls for sample separation, while ball-intra fusion [60] combined information within granular regions. VPGB [61] studied attribute reduction and classification with noisy labels, and GEAF [62] filtered label noise through adaptive granular-ellipsoid partitioning.

Granular representations have also been incorporated into deep learning. GBRAIN [63], G-GBC [64], and BNO-GB [65] applied granular regions to robust text classification, CNN classification, and open intent classification, respectively. On graphs, supervised granular ball coarsening [66] and GBGC [67] used granular regions to construct coarsened graphs, while MGGR [68] explored multi-granularity graph refinement for graph classification.

In supervised granular ball classification and sampling, label purity can guide whether a ball should be split [15, 16, 17]. Label-free geometric criteria are also used in granular ball computing [18]. GB-LCSS uses the latter type of criterion in a frozen graph representation: the partition is formed without observed labels, and each region

then determines which training nodes contribute to the local consensus. The balls organize supervision during training, while the deployed MLP predicts from node attributes.

These lines of work motivate combining label-free graph representations with label aggregation within local regions. GB-LCSS uses this combination to improve noisy supervision for an MLP while retaining the graph-free inference interface of methods such as Graph-MLP and GLNN.

3 Method

3.1 Problem setting

Consider a transductive attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X)$, where $\mathcal{V} = \{v_1, ..., v_N\}$ is the node set and $\mathcal{E}$ is the edge set. Let $A \in \{0,1\}^{N\times N}$ denote the adjacency matrix, and let $X = [x_1, ..., x_N]^\top \in \mathbb{R}^{N\times d}$ denote the node-attribute matrix. Each node belongs to one of C classes, and $\mathcal{V}_L \subset \mathcal{V}$ denotes the set of labeled training nodes. For each $v \in \mathcal{V}_L$, the observed label $\tilde{y}_v$ may be corrupted by an unknown noise process. GB-LCSS does not require prior knowledge of the noise type, noise rate, or label-transition matrix.

We consider a fixed-graph transductive setting. During training, the complete X and A, including the attributes and topology of test nodes, are available for label-free representation learning and target construction, while test labels remain hidden. After training, only the classifier is retained. Let f_θ denote the deployed MLP with parameters θ. For a target node v, its predictive class distribution is defined as

$$p_\theta(\cdot|x_v) = \text{softmax} \quad (f_\theta(x_v)). \tag{1}$$

Here, x_v is the attribute vector of node v and is the only node-specific input to the classifier at inference. The adjacency matrix A, neighboring nodes, the frozen representation, and the graph encoder are not used by the deployed classifier.

Table 1 summarizes the main notation.

Table 1. Main notation.

Symbol	Meaning
$\mathcal{G}, \mathcal{V}, \mathcal{E}, N$	Graph, node set, edge set, and number of nodes
A, X, x_v	Adjacency matrix, attribute matrix, and node attribute vector
$\mathcal{V}_L, \tilde{y}_v, C$	Labeled training nodes, observed label, and number of classes
$H, \bar{H}, d_h$	Frozen representation H, its row-normalized form $\bar{H}$, and dimension d_h
$\mathcal{B}, B_j, K$	Ball collection, nodes assigned to ball j, and final number of balls
$\mu_B, D(B), D_{\text{split}}(B)$	Center, mean radius, and size-weighted post-split dispersion
$B_{\max}, n_{\min}, \delta$	Ball budget, minimum child size, and required relative dispersion reduction
$T_j, T_{j,-v}$	Training nodes in ball j, before and after removing node v
$s_{\min}$	Minimum number of remaining label votes for using consensus
$p_{j,-v}, q_{j,-v}$	Empirical class distribution and local-consensus distribution
$r_{j,-v}, M_{j,-v}, m_{j,-v}$	Largest vote proportion, maximizing classes, and unique maximizing class
$\alpha_{j,-v}, \beta$	Consensus weight and sharpening parameter
t_v, e_c	Node-specific soft target and one-hot vector for class c
g_ψ, g_ϕ, f_θ	Frozen graph encoder, GCN teacher, and deployed MLP
Z^T, π_v, τ	Teacher logits, teacher predictive distribution, and distillation temperature
$\mathbb{I}(\cdot)$	Indicator: 1 if the condition holds, 0 otherwise

3.2 Label-free representation learning and granular ball construction

3.2.1 Label-free representation learning

GB-LCSS first builds a representation space that reflects node attributes and graph connectivity without using observed labels. We instantiate this representation-learning component with GRACE [14]. Given X and A, the component forms two stochastic graph views through edge dropping and feature masking and trains the encoder by aligning the representations of the same node across the two views. After pretraining, the encoder is frozen. We denote the encoder output before the GRACE projection head by the node-representation matrix H:

$$H = g_\psi(X, A) \in \mathbb{R}^{N \times d_h}. \tag{2}$$

Here, g_ψ denotes the pretrained graph encoder with frozen parameters ψ, d_h is the representation dimension, and the v-th row h_v is the representation of node v.

Before distance-based partitioning, we L2-normalize each node representation. Using $\varepsilon = 10^{-12}$ as a numerical safeguard, the normalized representation $\bar{h}_v$ is defined as

$$\bar{h}_v = \frac{h_v}{\max(\|h_v\|_2, \varepsilon)}. \tag{3}$$

The normalized representations are collected in $\bar{H} = [\bar{h}_1, ..., \bar{h}_N]^\top$, which is fixed during granular ball construction. Section 4.3.3 examines alternative representation sources and variation across GRACE pretraining seeds.

3.2.2 Granular ball construction

This stage groups nodes in the frozen representation $\bar{H}$ for subsequent label aggregation. We initialize the granular ball collection as $\mathcal{B} = \{\mathcal{V}\}$, so all nodes initially belong to one ball.

Definition 1. *For a node set B with normalized representations $\{\bar{h}_v : v \in B\}$, its center μ_B and mean radius $D(B)$ are defined as follows.*

$$\mu_B = \frac{1}{|B|}\sum_{v \in B} \bar{h}_v, \qquad D(B) = \frac{1}{|B|}\sum_{v \in B} \|\bar{h}_v - \mu_B\|_2. \tag{4}$$

The mean radius measures the average distance of the nodes from their center. A smaller value indicates a more compact group, so it can be used to assess whether a split improves local compactness without using labels. Unlike the purity criteria in supervised granular ball methods [15, 16, 17], this measure does not use observed labels.

Let $n_{\min}$ denote the minimum number of nodes allowed in each child ball. A current ball is eligible for splitting only when $|B| \geq 2n_{\min}$. Eligible balls are examined in descending order of $D(B)$, so more dispersed balls are considered first. For each candidate ball B, 2-means produces two child balls $B^{(1)}$ and $B^{(2)}$.

Definition 2. *The size-weighted post-split dispersion is defined as*

$$D_{\text{split}}(B) = \frac{|B^{(1)}|D(B^{(1)}) + |B^{(2)}|D(B^{(2)})}{|B|}. \tag{5}$$

Let $\delta \in (0,1)$ denote the minimum relative reduction in dispersion required for accepting a split. A candidate split is accepted only when both child balls contain at least $n_{\min}$ nodes and

$$D_{\text{split}}(B) \leq (1 - \delta)D(B). \tag{6}$$

Weighting by the sizes of the child groups makes the post-split value comparable with the parent value. A split is accepted only if both children satisfy the minimum size and their weighted mean radius decreases by at least a fraction δ. We accept the first candidate split that satisfies these conditions, then update the ball collection and reorder the remaining candidates.

Splitting stops when no valid candidate remains or when the number of balls reaches the prescribed positive integer budget $B_{\max}$. The resulting partition $\mathcal{B} = \{B_j\}_{j=1}^{K}$, with $K \le B_{\max}$, assigns every node to exactly one granular ball. Section 4.1.4 gives the experimental values of the partition parameters, and Algorithm 1 summarizes the procedure.

Algorithm 1. Recursive granular ball partitioning in the frozen representation space

Input: Normalized frozen representation $\bar{H}$, node set $\mathcal{V}$, parameters $B_{\max}, n_{\min}, \delta$
Output: Granular ball partition $\mathcal{B}$

$\mathcal{B} \leftarrow \{\mathcal{V}\}$
while $|\mathcal{B}| < B_{\max}$ **do**
  $\mathcal{B}_{\text{cand}} \leftarrow \{B \in \mathcal{B} : |B| \ge 2n_{\min}\}$, sorted by $D(B)$ in descending order
  accepted $\leftarrow$ false
  for each $B \in \mathcal{B}_{\text{cand}}$ **do**
    Partition B into $B^{(1)}$ and $B^{(2)}$ using 2-means
    if $\min(|B^{(1)}|, |B^{(2)}|) \ge n_{\min}$ *and* $D_{\text{split}}(B) \le (1-\delta)D(B)$ **then**
      $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{B\}) \cup \{B^{(1)}, B^{(2)}\}$
      accepted $\leftarrow$ true
      break
    end if
  end for
  if accepted = false **then**
    break
  end if
end while
return $\mathcal{B}$

3.3 Local-consensus supervision and MLP training

The partition defines which training nodes supply local evidence for each target. GB-LCSS first summarizes the observed labels of these nodes as a class distribution, then combines the resulting consensus with the current node's label. The resulting soft targets supervise the MLP.

3.3.1 Within-ball label consensus

Definition 3. *For a granular ball B_j, let $T_j = B_j \cap \mathcal{V}_L$ denote its training nodes. For each $v \in T_j$, the voting set $T_{j,-v}$ is defined as*

$$T_{j,-v} = T_j \setminus \{v\}. \tag{7}$$

Let $s_{\min}$ be a positive integer specifying the minimum number of remaining votes required to use local consensus. When $|T_{j,-v}| \ge s_{\min}$, we define the empirical class distribution $p_{j,-v}$ by

$$p_{j,-v,c} = \frac{1}{|T_{j,-v}|} \sum_{u \in T_{j,-v}} \mathbb{I}(\tilde{y}_u = c), \qquad c = 1, \ldots, C. \tag{8}$$

Excluding the current node prevents its observed label from reinforcing the consensus used to adjust it. Here, $\mathbb{I}(\tilde{y}_u = c)$ equals 1 if node u has observed label c and 0 otherwise. Equation (8) therefore counts the other training nodes labeled c and divides this count by the total number of remaining voting nodes. For efficient computation, class counts are accumulated once per ball. Subtracting the current node's vote gives the counts used for its target.

Definition 4. *Given the empirical class distribution $p_{j,-v}$, we define the largest class proportion $r_{j,-v}$ and the corresponding tie set $M_{j,-v}$ as*

$$M_{j,-v} = \{c: p_{j,-v,c} = r_{j,-v}\}, \qquad r_{j,-v} = \max_c p_{j,-v,c}. \tag{9}$$

Thus, $M_{j,-v}$ contains all classes attaining the largest number of votes. If $M_{j,-v}$ contains a single class, we denote it by $m_{j,-v}$.

Let $\beta \in [0,1]$ control the sharpening of a unique dominant class. We define the local-consensus distribution $q_{j,-v}$ as

$$q_{j,-v} = \begin{cases} (1-\beta)p_{j,-v} + \beta e_{m_{j,-v}}, & |M_{j,-v}| = 1, \\ p_{j,-v}, & |M_{j,-v}| > 1. \end{cases} \tag{10}$$

Here, e_c denotes the one-hot vector for class c. A unique dominant class is emphasized according to β, while a tied empirical distribution is retained without selecting a class from the tie. Because the current node is removed separately for each target, training nodes in the same ball can have different $p_{j,-v}$ and $q_{j,-v}$.

3.3.2 Consensus-weighted soft targets

The local-consensus distribution specifies the class probabilities supported by the regional votes. Its contribution to the final target is controlled separately by a weight computed from the original vote proportions before sharpening.

Definition 5. *The consensus weight $\alpha_{j,-v}$ is defined by normalizing the largest vote proportion $r_{j,-v}$ relative to the uniform class probability $1/C$:*

$$\alpha_{j,-v} = \max\left(0, \frac{r_{j,-v} - 1/C}{1 - 1/C}\right). \tag{11}$$

The weight $\alpha_{j,-v} \in [0,1]$ increases with the largest class proportion among the remaining votes. A uniform vote distribution gives $\alpha_{j,-v} = 0$, while complete agreement gives $\alpha_{j,-v} = 1$.

Definition 6. *The final training target t_v for $v \in T_j$ is defined as*

$$t_v = \begin{cases} (1-\alpha_{j,-v})e_{\tilde{y}_v} + \alpha_{j,-v} q_{j,-v}, & |T_{j,-v}| \geq s_{\min}, \\ e_{\tilde{y}_v}, & |T_{j,-v}| < s_{\min}. \end{cases} \tag{12}$$

When enough other training nodes are available in the ball, t_v combines the node's observed label with the local-consensus distribution. A larger proportion of votes for the most-voted class gives greater weight to $q_{j,-v}$. If fewer than $s_{\min}$ votes remain after excluding the current node, the target retains the observed one-hot label.

Equations (7)-(12) use only the observed labels of training nodes. Validation and test labels are not used in target construction. Algorithm 2 summarizes the target construction.

Algorithm 2. Granular ball soft-target construction

Input: Granular ball set $\mathcal{B}$, noisy labels $\{\tilde{y}_v \mid v \in \mathcal{V}_L\}$, number of classes C, parameters $s_{\min}, \beta$
Output: Soft targets for training nodes $\{t_v \mid v \in \mathcal{V}_L\}$

Initialize $t_v \leftarrow e_{\tilde{y}_v}$ for every $v \in \mathcal{V}_L$
for each $B_j \in \mathcal{B}$ **do**
  $T_j \leftarrow B_j \cap \mathcal{V}_L$
  $n_j \leftarrow \sum_{u \in T_j} e_{\tilde{y}_u}$
  for each $v \in T_j$ **do**
    $n_{j,-v} \leftarrow n_j - e_{\tilde{y}_v}$
    if $\sum_c n_{j,-v,c} < s_{\min}$ **then**
      continue
    end if
    $p_{j,-v} \leftarrow n_{j,-v} / \sum_c n_{j,-v,c}$
    $r_{j,-v} \leftarrow \max_c p_{j,-v,c}$
    $M_{j,-v} \leftarrow \{c : p_{j,-v,c} = r_{j,-v}\}$
    if $|M_{j,-v}| = 1$ **then**
      $q_{j,-v} \leftarrow (1-\beta)p_{j,-v} + \beta e_{m_{j,-v}}$, where $m_{j,-v} \in M_{j,-v}$
    else
      $q_{j,-v} \leftarrow p_{j,-v}$
    end if
    $\alpha_{j,-v} \leftarrow \max\left(0, \frac{r_{j,-v}-1/C}{1-1/C}\right)$
    $t_v \leftarrow (1-\alpha_{j,-v})e_{\tilde{y}_v} + \alpha_{j,-v} q_{j,-v}$
  end for
end for
return $\{t_v \mid v \in \mathcal{V}_L\}$

3.3.3 MLP training

The node-specific targets in Eq. (12) supervise the two-layer MLP f_θ. Its hidden linear layer is followed by ReLU, batch normalization, and dropout, and its output layer produces class logits. We define the GB-LCSS training loss $\mathcal{L}_{\text{LCSS}}$ as the soft cross-entropy over $\mathcal{V}_L$:

$$\mathcal{L}_{\text{LCSS}} = -\frac{1}{|\mathcal{V}_L|}\sum_{v \in \mathcal{V}_L}\sum_{c=1}^{C} t_{vc} \log p_\theta(c|x_v). \tag{13}$$

Here, t_{vc} is the probability assigned to class c by the target t_v, and $p_\theta(c|x_v)$ is the MLP prediction defined in Eq. (1).

After training, only f_θ is retained for inference on node attributes.

3.4 Analysis of local-consensus supervision

The benefit of local consensus depends on whether the other training nodes provide evidence for the current node's true class. We first analyze the reliability of this evidence under independent symmetric label noise, then relate it to the soft target in Eq. (12). The analysis conditions on the frozen representation, granular ball assignments, training-node set, and clean labels.

Consider a training node v. For analysis, let y_v be its clean label and $s_v = |T_{j,-v}| \geq s_{\min}$ the number of remaining votes. Let $p_{v,c}^{\text{clean}}$ be the proportion of these nodes whose clean label is c, and define the margin of the current node's class as

$$p_{v,c}^{\text{clean}} = \frac{1}{s_v}\sum_{u \in T_{j,-v}} \mathbb{I}(y_u = c), \qquad \Delta_v = p_{v,y_v}^{\text{clean}} - \max_{c \neq y_v} p_{v,c}^{\text{clean}}. \tag{14}$$

Proposition 1. Suppose that $C \geq 2$, $\Delta_v > 0$, and each voting label is independently flipped with probability η to a uniformly chosen incorrect class. If $0 \leq \eta < (C-1)/C$, let $\gamma = 1 - C\eta/(C-1) > 0$. Then

$$\Pr\left(\max_{c \neq y_v} p_{j,-v,c} \geq p_{j,-v,y_v}\right) \leq \min\left\{1, (C-1)\exp\left(-\frac{s_v \gamma^2 \Delta_v^2}{2}\right)\right\}. \tag{15}$$

The event includes a tie for the most votes. Equation (10) preserves the maximizing class set of $p_{j,-v}$, so the same bound applies to $q_{j,-v}$. With the class count, noise rate, and positive margin fixed, the exponential term decreases as more independent votes are available. The margin also matters: adding nodes from competing classes can weaken the evidence for the current node. Appendix H.1 gives the proof.

The final target combines this evidence with the observed label. Write $z = e_{\tilde{y}_v}$, $q = q_{j,-v}$, $\alpha = \alpha_{j,-v}$, and $L_v(r) = \| r - e_{y_v} \|_2^2$ for the squared probability error of a target vector r. The target $t_v = (1-\alpha)z + \alpha q$ satisfies the exact identity

$$L_v(t_v) = (1-\alpha)L_v(z) + \alpha L_v(q) - \alpha(1-\alpha)\| z - q \|_2^2. \tag{16}$$

Equation (16) relates soft-target error to the errors of the observed label and the local consensus, together with their disagreement. It applies directly to the consensus weight and tie-handling rule in Eqs. (9)–(12). Combining these sources can correct corrupted labels and can also perturb initially correct labels. The overall change in target error depends on the balance of these effects. Appendix H.2 derives the identity and a sufficient condition for lower conditional expected target error under symmetric noise. Section 4.3.1 and Appendix I.2 examine the two effects empirically.

3.5 Graph-teacher distillation extension

GB-LCSS directly supervises the MLP on the labeled training nodes $\mathcal{V}_L$. In the extension, a GCN teacher is trained on the same soft targets over $\mathcal{V}_L$, and its predictions for all nodes supervise the MLP student.

For each $v \in \mathcal{V}_L$, we define the teacher target t_v^T using the target defined in Eq. (12):

$$t_v^T = t_v, v \in \mathcal{V}_L. \tag{17}$$

Let g_ϕ denote a two-layer GCN teacher with parameters ϕ, and let $Z^T \in \mathbb{R}^{N \times C}$ denote its class-logit matrix. Given the original node attributes X and adjacency matrix A, the teacher output is

$$Z^T = g_\phi(X, A). \tag{18}$$

Let Z_v^T denote the row of Z^T corresponding to node v. We define the teacher loss $\mathcal{L}_T$ over the labeled training nodes as

$$\mathcal{L}_T = -\frac{1}{|\mathcal{V}_L|} \sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} t_{vc}^T \operatorname{logsoftmax}(Z_v^T)_c. \tag{19}$$

The loss in Eq. (19) trains the graph teacher to match the GB-LCSS target probabilities on $\mathcal{V}_L$. The normalized representation $\bar{H}$ is used to construct the granular ball partition from which these targets are derived; the teacher itself receives X and A as inputs.

After checkpoint selection, the teacher is frozen. Let $\tau > 0$ denote the distillation temperature. For every $v \in \mathcal{V}$, we define the temperature-scaled teacher prediction π_v as

$$\pi_v = \operatorname{softmax}\left(\frac{Z_v^T}{\tau}\right). \tag{20}$$

Unlike t_v, which is constructed only for labeled training nodes, π_v is available for every node in $\mathcal{V}$.

The MLP student f_θ is trained on all N nodes using π_v as its sole distillation target. We define the student distillation loss $\mathcal{L}_{\mathrm{KD}}$ as

$$\mathcal{L}_{\mathrm{KD}} = -\frac{\tau^2}{N} \sum_{v \in \mathcal{V}} \sum_{c=1}^{C} \pi_{vc} \operatorname{logsoftmax}\left(\frac{f_\theta(x_v)}{\tau}\right)_c. \tag{21}$$

After distillation, only the MLP student is retained for inference on node attributes.

3.6 Computational costs

Relative to direct MLP training, GB-LCSS adds graph contrastive pretraining, granular ball partitioning, and soft-target generation. The cost of graph contrastive pretraining depends on the encoder configuration and training schedule; Section 4.4.2 reports the measured representation-learning and grouping costs. The analysis below focuses on the subsequent construction stages and MLP training and inference.

Running 2-means for I iterations on a node subset S costs $\mathcal{O}(I|S|d_h)$. Candidate balls examined within one pass are disjoint, so their combined size is at most N. Since at most $B_{\max} - 1$ splits can be accepted, a conservative upper bound on the granular ball partitioning cost is

$$O(IB_{\max}Nd_h). \tag{22}$$

For fixed I and $B_{\max}$, this bound is linear in N and d_h. Storing the frozen representation $\bar{H}$ requires $\mathcal{O}(Nd_h)$ space, while node assignments, ball centers, and class counts require an additional $\mathcal{O}\left(N + B_{\max}(d_h + C)\right)$ space. Reusing these class counts, soft-target generation takes $\mathcal{O}(|\mathcal{V}_L|C)$ time. The current implementation stores an $N \times C$ target matrix, requiring $\mathcal{O}(NC)$ space.

After target construction, GB-LCSS trains the same two-layer MLP used as the common baseline. Let F denote its hidden width and T_S its number of student-training epochs. The loss in Eq. (13) involves only the training nodes, but the current full-batch implementation evaluates all N nodes in each of the T_S epochs, at a cost of $\mathcal{O}\left(T_S N(dF + FC)\right)$. For a single target node, the MLP inference cost is

$$O(dF + FC). \tag{23}$$

Storing the deployed MLP parameters also requires $\mathcal{O}(dF + FC)$ space. Inference uses only the MLP and the target node's attributes, so its per-node cost is independent of the number of graph edges $|\mathcal{E}|$.

The distillation extension additionally requires GCN teacher training. Its MLP student learns from teacher predictions over all nodes and retains the same deployed architecture as GB-LCSS without distillation.

4 Experiments

4.1 Experimental setup

4.1.1 Datasets

We evaluate GB-LCSS on five public datasets covering citation, co-authorship, and co-purchase graphs: Cora and CiteSeer [69], together with CoraFull, Coauthor-CS, and Amazon-Photo [70]. Table 2 summarizes their statistics, with edges counted as unique undirected pairs after self-loop removal and training-node counts based on the 20% split. Edge homophily is the proportion of unique undirected edges joining nodes of the same class. Six additional datasets are evaluated under uniform 0.4 (Appendix G). Appendices A–I are provided in the Supplementary Material.

Table 2. Dataset statistics.

Dataset	Nodes	Undirected edges	Features	Classes	Training nodes	Edge homophily
Cora	2,708	5,278	1,433	7	541	0.810
CiteSeer	3,327	4,552	3,703	6	665	0.736
CoraFull	19,793	63,421	8,710	70	3,958	0.567
Coauthor-CS	18,333	81,894	6,805	15	3,666	0.808
Amazon-Photo	7,650	119,081	745	8	1,530	0.827

4.1.2 Data splits and label noise

All comparisons use a fixed-graph transductive protocol. Complete node attributes and adjacency, including those of test nodes, are available as unlabeled context during representation learning and partitioning. For seeds

0–4, the NoisyGL percentage-split routine samples nodes without class stratification into training, validation, and test subsets of sizes $\lfloor 0.2N \rfloor$, $\lfloor 0.1N \rfloor$, and $\lfloor 0.7N \rfloor$. Because the three subset sizes are obtained by integer flooring, at most two nodes may remain outside the three subsets. Methods share the same split and noisy training labels for each dataset and seed. Noise is injected into training labels only. Clean validation labels are used for checkpoint selection, and test labels are used for final evaluation. Appendix C examines corrupted validation labels.

Uniform noise uses $P_{ii} = 1 - \rho$ and $P_{ij} = \rho/(C-1)$, with $\rho \in \{0.2, 0.4, 0.6\}$. Pair noise uses $\rho = 0.4$ and changes class i to $((i-2) \bmod C) + 1$ with probability ρ, leaving it unchanged with probability $1 - \rho$. Instance-dependent noise follows the NoisyGL generator with a configured mean flip rate of 0.4 and norm_std $= 0.1$.

4.1.3 Compared methods

The first comparison group comprises seven noisy-label baselines: MLP+CE, Small-loss, Co-teaching, JoCoR, SCE, ELR, and LogitClip. They use the same deployed two-layer MLP and labeled training nodes as GB-LCSS. GB-LCSS additionally uses graph structure during training. Small-loss, Co-teaching, and JoCoR receive the known synthetic noise rate for their forget-rate schedules; the other methods in this group and GB-LCSS do not. Small-loss selects samples by per-sample cross-entropy. Co-teaching and JoCoR train two independently initialized MLPs but evaluate one network without ensembling.

The second group includes PosteL [54], ALS [55], SALS [56], Graph-MLP [3], and GSDN [42], which use graph information during training and receive only node attributes at inference in our protocol. PosteL integrates the official target-construction code with iterative pseudo-labeling disabled. Graph-MLP is ported to the common protocol, while ALS, SALS, and GSDN use mechanism adaptations. Appendix F specifies the implementation basis, adaptations, and parameter sources.

The third group compares graph-to-MLP transfer methods: a GCN(X)+CE response-distillation baseline based on GLNN [4], FF-G2M [6], KRD [46], VQGraph [47], and a GCN teacher trained on GB-LCSS targets (Section 3.5). FF-G2M is reimplemented with common backbones and its low- and high-frequency objectives. The KRD and VQGraph mechanism adaptations retain reliable-knowledge selection and soft-codebook mechanisms, respectively. For each seed, we retrain the teacher and student for every route using a common MLP configuration.

4.1.4 Implementation details

Datasets are loaded through the NoisyGL interface to PyTorch Geometric. Graph contrastive pretraining uses row-normalized attributes, whereas downstream models use the attributes supplied by NoisyGL. Graph-convolutional models use the supplied adjacency, with normalization and self-loops handled within their graph-convolution layers.

The noisy-label baselines and GB-LCSS use a two-layer MLP with hidden width 64 and dropout 0.5. We train for 200 epochs using Adam with a learning rate of 0.01 and weight decay 5×10^{-4}, and select the checkpoint with the highest validation accuracy. Teacher checkpoints use the same selection criterion.

GB-LCSS uses $B_{\max} = 64$, $n_{\min} = 32$, $\delta = 0.05$, $s_{\min} = 2$, and $\beta = 0.5$ as the default settings. Recursive 2-means uses Euclidean distance, k-means++ initialization [71], and 10 initializations per candidate split, with randomization controlled by the downstream seed. The graph encoder uses the fixed GRACE configurations listed in Appendix F and is frozen after pretraining. The distillation extension uses temperature $\tau = 1$. Complete parameters and their sources appear in Appendix F.

4.1.5 Evaluation protocol

Unless stated otherwise, test-node classification accuracy is reported as percentages, with mean $\pm$ sample standard deviation over five paired seeds. Bold marks the highest mean within each dataset and noise setting; in Table 4, underlining marks the second-highest mean. Each dataset uses one fixed pretrained representation H; the

seeds vary the split, noise realization, downstream initialization, and 2-means initialization. Section 4.3.3 separately examines variation across GRACE pretraining seeds.

We assess same-seed accuracy differences using two-sided paired t-tests and 95% confidence intervals. To account for multiple comparisons, Holm correction [72] is applied separately to each set of 25 tests against the following reference methods: random size-matched grouping, MLP+CE, and JoCoR. Appendix B gives the full paired results and calculation details.

4.2 Comparative results

4.2.1 Comparison with noisy-label baselines

Table 3. Test accuracy of matched-MLP methods under uniform 0.4 noise.

Method	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
MLP+CE	44.24±2.62	43.50±3.02	31.98±0.98	88.42±0.62	73.90±1.80
Small-loss	51.95±1.97	49.70±3.07	33.04±0.50	89.06±0.88	77.25±1.62
Co-teaching	46.83±1.78	46.24±2.89	34.39±0.24	88.80±0.97	78.21±1.00
JoCoR	51.72±1.30	49.91±3.37	35.70±0.32	89.38±0.61	79.57±0.53
SCE	43.60±1.86	42.09±3.47	29.29±0.45	88.37±0.53	76.08±3.11
ELR	44.28±1.38	43.17±3.95	32.14±0.67	88.29±0.73	73.93±2.12
LogitClip	47.50±1.50	46.57±3.52	34.69±1.17	88.40±0.21	74.98±2.38
GB-LCSS	**57.49±0.76**	**55.24±2.04**	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

GB-LCSS attains the highest mean accuracy on all five datasets under uniform 0.4, exceeding the strongest baseline by 1.33–5.54 percentage points (Table 3). Across all five noise settings, it attains or ties for the highest mean in 24 of 25 settings, with average gains of 7.74 points over MLP+CE and 3.46 points over JoCoR (Fig. 2). On Coauthor-CS under uniform 0.2, JoCoR is higher by 0.27 points. After correcting for multiple comparisons with the Holm procedure, the paired differences remain significant in 24 of 25 settings against MLP+CE and 11 of 25 against JoCoR (Appendix B). Appendix A gives the complete results.

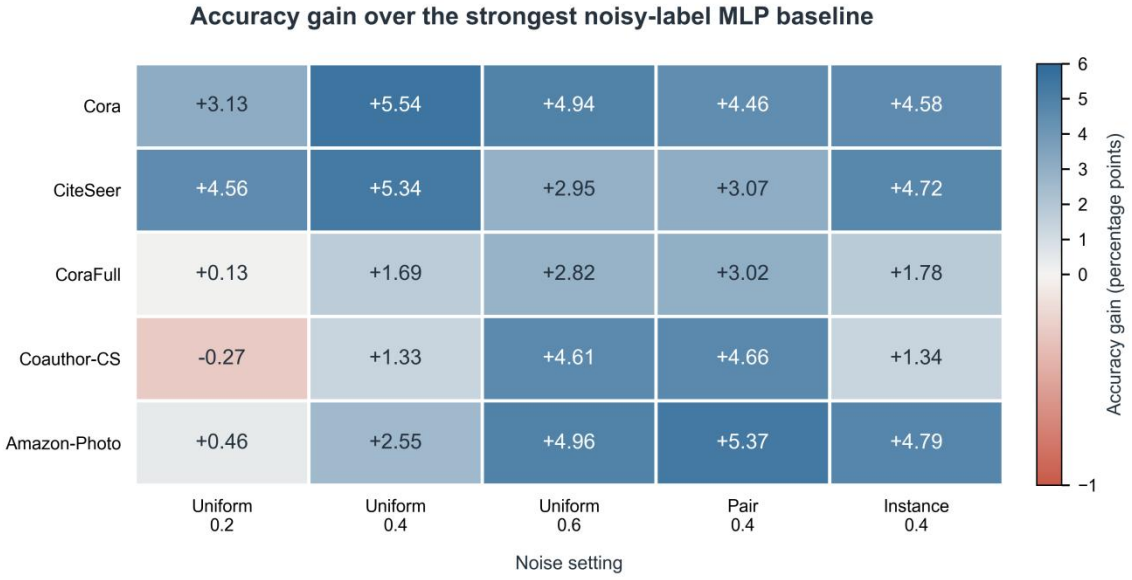


Fig. 2. Accuracy gain of GB-LCSS over the strongest noisy-label baseline with the same deployed MLP. Each cell is the five-seed mean accuracy of GB-LCSS minus the highest mean among the seven baselines under the same dataset and noise setting, in percentage points. Blue and red denote positive and negative gains, respectively.

Fig. 3 shows how accuracy changes with the uniform noise rate. From 0.2 to 0.6, the decrease in accuracy for GB-LCSS is smaller than that for MLP+CE and JoCoR on four of the five datasets.

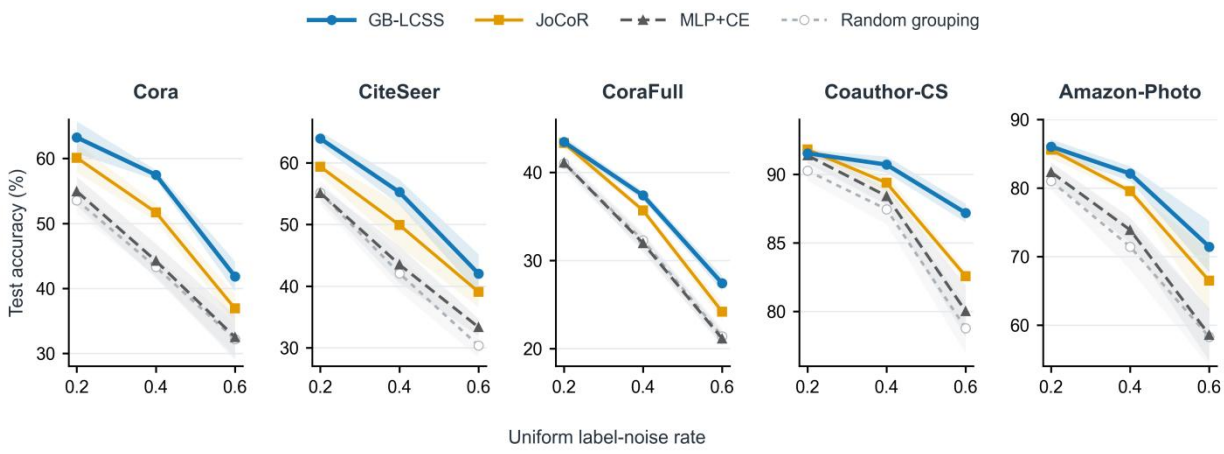


Fig. 3. Test accuracy under uniform label noise. Lines show the means over five paired seeds, and shaded regions show sample standard deviations; the y-axis range differs across datasets. Random size-matched grouping randomizes node membership while preserving the number and size distribution of granular balls.

On the six additional datasets under uniform 0.4, GB-LCSS exceeds random size-matched grouping on all six and the strongest matched-backbone noisy-label baseline on three. Appendix G gives the full results and the exploratory evaluation of CiteSeerFull under the other four noise settings.

4.2.2 Comparison with graph-assisted training methods

Table 4 compares methods that use graph information to construct soft targets or guide MLP training.

Table 4. Test accuracy of graph-assisted training methods under uniform 0.4 noise.

Method	Use of graph structure in training	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
PosteL	Posterior soft labels	47.09±1.81	45.84±2.48	36.51±1.01	84.74±1.51	80.59±0.94
ALS	Neighborhood label smoothing	45.20±1.24	43.77±2.76	31.91±1.05	88.55±0.86	73.41±2.75
SALS	Structure-aware label smoothing	48.27±2.50	47.20±1.69	37.06±0.78	88.85±0.82	77.86±0.79
Graph-MLP	Multi-hop neighborhood-contrastive objective	**63.23±3.07**	**57.23±1.85**	37.05±0.91	88.95±0.43	77.05±1.24
GSDN	Neighborhood self-distillation	55.97±4.29	50.36±3.39	35.80±0.66	86.55±0.78	73.31±2.00
GB-LCSS	Label consensus within granular balls	57.49±0.76	55.24±2.04	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

Averaged across the 25 settings, GB-LCSS exceeds the protocol-adapted PosteL, ALS, and SALS implementations by 5.69, 7.71, and 4.69 percentage points, respectively. The corresponding differences are positive in 24, 25, and 24 settings (Appendix E).

Under uniform 0.4, GB-LCSS has higher mean accuracy than Graph-MLP on CoraFull, Coauthor-CS, and Amazon-Photo, while Graph-MLP has higher mean accuracy on Cora and CiteSeer. GB-LCSS also has higher mean accuracy than the adapted GSDN implementation on all five datasets under this noise setting.

4.3 Ablation and mechanism analysis

4.3.1 Effect of local-consensus targets

We first test whether the identities of the nodes grouped together matter for supervision. Random size-matched grouping shuffles node membership while preserving group sizes. The soft-target rule is unchanged, so this comparison tests whether preserving the group-size distribution alone is sufficient.

Uniform and prior smoothing replace regional consensus with a uniform class distribution or the class frequencies of the observed training labels, respectively. These controls test whether simply softening the targets provides the same benefit. A further variant includes the current node's label in the vote while retaining the same tie-handling rule. Table 5 reports these comparisons under uniform 0.4; its last column gives GB-LCSS minus the variant including the node's own vote, in percentage points.

Table 5. Target-construction ablations under uniform 0.4 noise.

Dataset	GB-LCSS	Own vote included	Random size-matched grouping	Uniform smoothing	Prior smoothing	Gain from excluding own vote
Cora	**57.49±0.76**	56.28±1.54	43.31±1.40	44.69±2.90	44.89±2.38	+1.21
CiteSeer	**55.24±2.04**	53.52±2.71	42.08±2.88	43.69±3.31	43.43±2.99	+1.72
CoraFull	**37.39±0.51**	37.30±0.57	32.30±0.85	32.59±1.12	32.63±0.84	+0.09
Coauthor-CS	**90.71±0.59**	90.62±0.47	87.46±1.07	88.25±0.80	88.04±0.73	+0.09
Amazon-Photo	**82.12±1.08**	81.33±0.70	71.45±3.29	74.18±1.72	74.72±1.44	+0.79

GB-LCSS exceeds random size-matched grouping in all 25 settings, with a mean accuracy difference of 8.69 percentage points. This indicates that group membership matters for local-consensus target construction. Under uniform 0.4, GB-LCSS also has higher mean accuracy than uniform and class-prior smoothing on all five datasets, supporting the use of labels from training nodes within each region rather than generic smoothing distributions.

Excluding the node's own vote raises mean accuracy on all five datasets under uniform 0.4 and, averaged over the five noise settings, on every dataset. A complementary control uses clean training labels: GB-LCSS improves on MLP+CE only on CiteSeer and has lower accuracy on the other four datasets (Appendix D).

We then assess the training targets themselves, separately from classification accuracy. Fig. 4 uses the multiclass Brier score [73] to measure their squared probability error against the clean training labels, which are used for evaluation only. For a target matrix Y^{tar}, the score is

$$\mathrm{BS}(Y^{\mathrm{tar}}) = \frac{1}{|\mathcal{V}_L|} \sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} (Y_{vc}^{\mathrm{tar}} - \mathbf{1}[y_v = c])^2 ,$$

where Y_{vc}^{tar} is the target probability assigned to class c for node v, and y_v is the clean label of training node v. Lower scores indicate smaller squared probability error. Each of the 25 points, averaged over five seeds, falls below the equality line: GB-LCSS reduces the score by 0.06–0.51 relative to the observed one-hot targets, which assign probability 1 to the observed class and 0 to all other classes. The largest reductions occur on Coauthor-CS and Amazon-Photo.

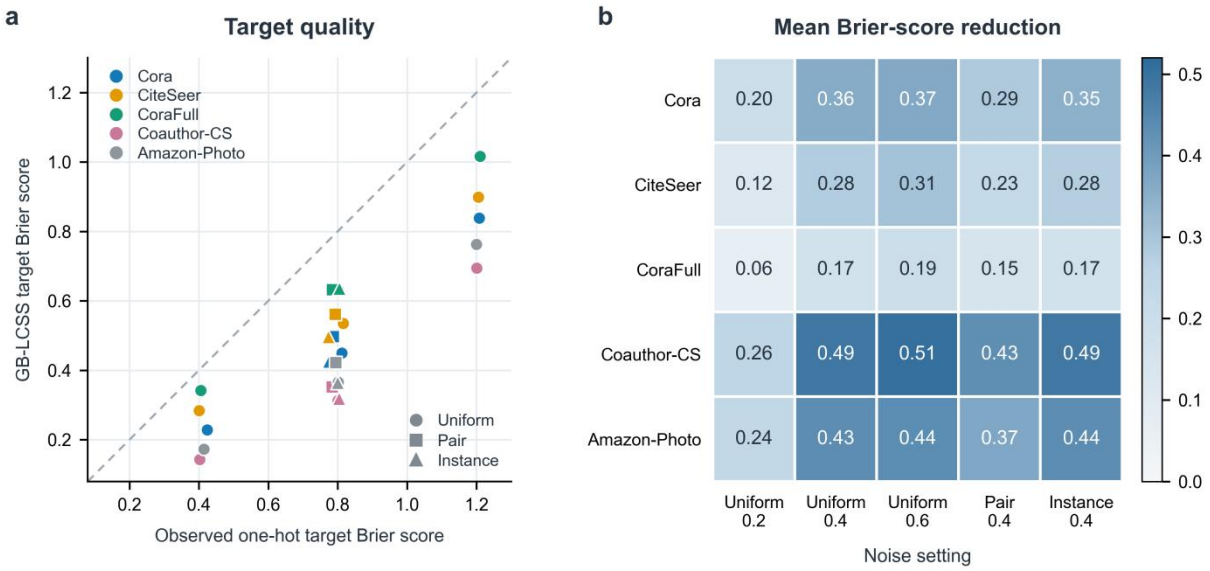


Fig. 4. Brier score of the GB-LCSS targets and of the observed one-hot targets against the clean training labels. (a) The horizontal and vertical axes show the multiclass Brier scores of the observed one-hot targets and the GB-LCSS targets, respectively. Each marker is the five-seed mean for one dataset–noise setting; colors identify datasets, and circles, squares, and triangles denote uniform, pair, and instance-dependent noise. The dashed line denotes equality. (b) Each cell is the observed-target score minus the GB-LCSS-target score; positive values indicate lower squared probabilistic error for the soft targets.

Across the five datasets under uniform 0.4 noise, we also separate the reduction in error on corrupted labels from the increase on initially correct labels (Appendix I.2). The reduction exceeds the increase in all 25 dataset–seed runs.

4.3.2 Sensitivity to the grouping rule

To assess the effect of granular ball grouping, we replace it with kNN neighborhoods or k-means clusters while keeping the frozen representation and target-construction rule in Eqs. (7)–(12) unchanged. For each seed, kNN excludes the query node and matches its neighborhood size to the mean number of training votes per ball. k-means uses the same number of groups as the granular ball partition and likewise excludes the current node's own vote from the local consensus. Table 6 reports the comparison under uniform 0.4, with differences defined as alternative-grouping accuracy minus granular ball accuracy, in percentage points.

Table 6. Grouping comparison under uniform 0.4 noise.

Dataset	Granular ball	kNN	kNN - GB	k-means	k-means - GB
Cora	**57.49±0.76**	57.21±1.19	−0.27	56.58±1.23	−0.91
CiteSeer	55.24±2.04	56.17±3.24	+0.94	**56.18±1.78**	+0.94
CoraFull	37.39±0.51	**40.50±0.51**	+3.11	39.26±0.48	+1.87
Coauthor-CS	90.71±0.59	90.83±0.57	+0.12	**91.01±0.64**	+0.29
Amazon-Photo	82.12±1.08	**82.23±1.20**	+0.11	81.68±0.66	−0.44

Granular ball grouping is within one percentage point of both alternatives on four datasets; on CoraFull, kNN is higher by 3.11 points. The comparison shows that the local-consensus target construction can be used with different region definitions, while no grouping rule is best on every dataset.

Section 4.4.2 compares the construction time and storage of the three grouping rules.

4.3.3 Effect of the representation space

The representation determines which nodes are grouped together and which labels contribute to each target. Table 7 compares GRACE, raw node attributes X, and two-step propagated attributes A^2X on all five datasets under uniform 0.4 noise. Each condition was rerun with five paired seeds and the same splitting parameters

(Appendix I.1). For A^2X, the original adjacency matrix multiplies the attributes twice, with no added self-loops or adjacency normalization. The resulting node vectors are L2-normalized as in Eq. (3).

Table 7. Classification accuracy (%) and ball counts for three representation spaces under uniform 0.4 noise. Accuracy is mean ± sample standard deviation over five paired seeds; ball counts give the range across seeds.

Dataset	GRACE accuracy	Raw X accuracy	A²X accuracy	GRACE balls	Raw X balls	A²X balls
Cora	**57.52 ± 0.71**	44.37 ± 2.31	44.37 ± 2.31	52–53	1	1
CiteSeer	**55.24 ± 2.04**	42.90 ± 2.50	42.90 ± 2.50	63–64	1	1
CoraFull	**37.22 ± 0.66**	31.86 ± 1.05	31.86 ± 1.05	64	1	1
Coauthor-CS	**90.68 ± 0.58**	87.63 ± 1.10	89.20 ± 0.67	64	1	50–59
Amazon-Photo	**82.32 ± 0.55**	73.38 ± 1.13	80.94 ± 1.75	64	1	64

Raw X produced one ball on every dataset and seed. In this case, each training node obtains its consensus from the labels of all other training nodes in the graph. On Cora, CiteSeer, and CoraFull, A^2X also produced one ball, giving the same targets and accuracy as raw X. On Coauthor-CS and Amazon-Photo, propagation enabled 50–59 and 64 balls, respectively, and improved accuracy over raw X. GRACE achieved the highest mean accuracy among the three representations on all five datasets, exceeding A^2X by 1.47 and 1.38 percentage points on the latter two. Thus, simple propagation can support local grouping on some datasets, while the learned representation gave the most effective supervision in this comparison.

We separately examined GRACE pretraining variability on Cora, CiteSeer, and CoraFull, using three pretrained representations and three downstream seeds for each. The resulting partitions differed: their adjusted Rand index [74], a measure of agreement between partitions, ranged from 0.247 to 0.365. Classification accuracy, however, was relatively stable. After averaging over downstream seeds, its standard deviation across pretraining seeds was 0.13, 0.45, and 0.60 percentage points on Cora, CiteSeer, and CoraFull, respectively. GB-LCSS exceeded MLP+CE for every pretraining seed. Thus, the tested representations yielded different node assignments but similarly effective supervision under uniform 0.4 noise.

4.3.4 Sensitivity to construction and target parameters

We examine how the parameters affect the formation of local groups, the availability of label evidence, and its weight in the target. Table I.1 varies one parameter at a time on all five datasets under uniform 0.4 noise, using fixed pretrained representations and five paired seeds.

The ball budget and splitting threshold determine how finely the nodes are grouped. On CoraFull, reducing $B_{\max}$ from 64 to 32 lowered mean accuracy by 2.10 percentage points; increasing it to 128 raised accuracy by 1.57 points. The larger budget left Cora unchanged and slightly lowered accuracy on Coauthor-CS and Amazon-Photo. Raising δ from 0.05 to 0.10 prevented splitting on the first four datasets, with losses of 3.04–13.15 points. Amazon-Photo still formed 64 balls and changed by −0.18 points. Lowering δ to 0.01 changed accuracy by at most 0.04 points across the five datasets.

The voting threshold $s_{\min}$ controls how many other labeled nodes are needed to use consensus. Raising it from 2 to 10 reduced the fraction of training nodes receiving a positive consensus weight to 56.53% on Cora and 58.32% on CiteSeer, accompanied by accuracy losses of 7.27 and 5.22 points. Coverage remained at least 89.96% on the other three datasets, where accuracy changed by at most 0.38 points. The value of a stricter voting threshold therefore depends on the available labels within each group.

Giving more weight to the most-voted class also had dataset-specific effects. Increasing β from 0.5 to 1 raised mean accuracy on four datasets, most noticeably on CiteSeer (+1.74 points), and lowered it by 0.14 points on CoraFull. Changes to $n_{\min}$ likewise had mixed effects. Several configurations improved on the common default, with different settings yielding the highest mean on different datasets. The clearest losses arose when local groups failed to form or too few nodes could use consensus.

4.4 Graph-teacher extension and computational cost

4.4.1 Graph-teacher distillation

We next evaluate the graph-teacher extension from Section 3.5. Table 8 compares direct MLP training and graph-to-MLP distillation on the five datasets under uniform 0.4 and 0.6. The first row uses direct MLP training on labeled nodes; the remaining rows use graph-to-MLP distillation with method-specific objectives and node selection. All routes deploy the same two-layer MLP receiving only node attributes.

Compared with the GCN(X)+CE baseline, the GCN teacher trained on GB-LCSS targets uses the soft target in Eq. (17) in place of the observed one-hot label. The remaining distillation routes and their implementations are described in Section 4.1.3 and Appendix F.

Table 8. Direct MLP training and graph-to-MLP distillation under uniform noise.

(a) Uniform 0.4

Training route	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
GB-LCSS, direct MLP training	57.49±0.76	55.24±2.04	37.39±0.51	90.71±0.59	82.12±1.08
GCN(X)+CE teacher	77.95±2.62	66.09±1.38	**58.74±0.82**	93.20±0.09	89.86±0.77
FF-G2M	77.65±2.97	65.80±0.49	56.58±0.93	91.58±0.24	84.82±0.84
KRD	71.77±2.20	64.96±1.03	47.12±0.95	91.39±0.49	85.17±0.94
VQGraph	78.89±2.34	65.64±1.72	58.52±0.77	93.25±0.09	**90.54±0.85**
GCN teacher trained on GB-LCSS targets	**81.35±1.05**	**69.27±1.57**	57.29±0.24	**93.83±0.21**	90.40±0.77

(b) Uniform 0.6

Training route	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
GB-LCSS, direct MLP training	41.86±2.30	42.02±3.14	27.43±0.89	87.19±0.69	71.45±3.75
GCN(X)+CE teacher	65.50±3.28	53.30±4.42	51.63±2.02	91.13±0.58	84.80±1.48
FF-G2M	63.63±2.16	52.32±4.20	45.78±2.90	86.50±1.01	73.08±2.82
KRD	61.28±1.17	54.17±3.14	41.61±2.43	89.80±0.29	80.30±0.51
VQGraph	65.75±2.72	53.34±4.47	52.01±0.65	90.71±1.24	84.77±2.19
GCN teacher trained on GB-LCSS targets	**71.34±3.91**	**58.93±4.62**	**52.32±1.26**	**92.40±0.65**	**86.81±1.94**

Training the GCN teacher with GB-LCSS targets instead of observed labels raises the distilled student's mean accuracy in nine of ten settings. The full extension, combining GCN teacher training with distillation over all nodes, has higher mean accuracy than direct MLP training in all ten settings, by 3.12–23.86 points under uniform 0.4 and 5.21–29.48 points under uniform 0.6.

The student distilled from the GB-LCSS-trained teacher has the highest mean among the five distillation routes in eight of ten settings; on CoraFull and Amazon-Photo under uniform 0.4, GCN(X)+CE and VQGraph are higher by 1.45 and 0.14 points.

4.4.2 Computational cost

GB-LCSS retains the same deployed MLP as MLP+CE; its additional costs arise during representation learning and target construction. The timing and GPU-memory measurements were obtained on an AMD Radeon RX 7700 XT workstation running Windows 10 (build 19045), with Python 3.12.10, PyTorch 2.9.1+rocm7.2.1, and PyTorch Geometric 2.7.0. Representative measurements on Cora, CiteSeer, and CoraFull show GRACE pretraining times of 0.07, 0.13, and 74.48 minutes and peak PyTorch-allocated GPU tensor memory of 0.40, 0.67, and 12.73 GiB, respectively. Times are averaged over three pretraining seeds, and memory values are the largest peaks recorded by torch.cuda.max_memory_allocated. Each frozen representation is reused across noise settings and downstream seeds.

Fig. 5 compares the construction time and storage of the three grouping rules used for direct MLP training. Granular ball grouping uses less storage than exact kNN on CoraFull, Coauthor-CS, and Amazon-Photo, and has the shortest construction time on three of the five datasets in the seed-0 measurements.

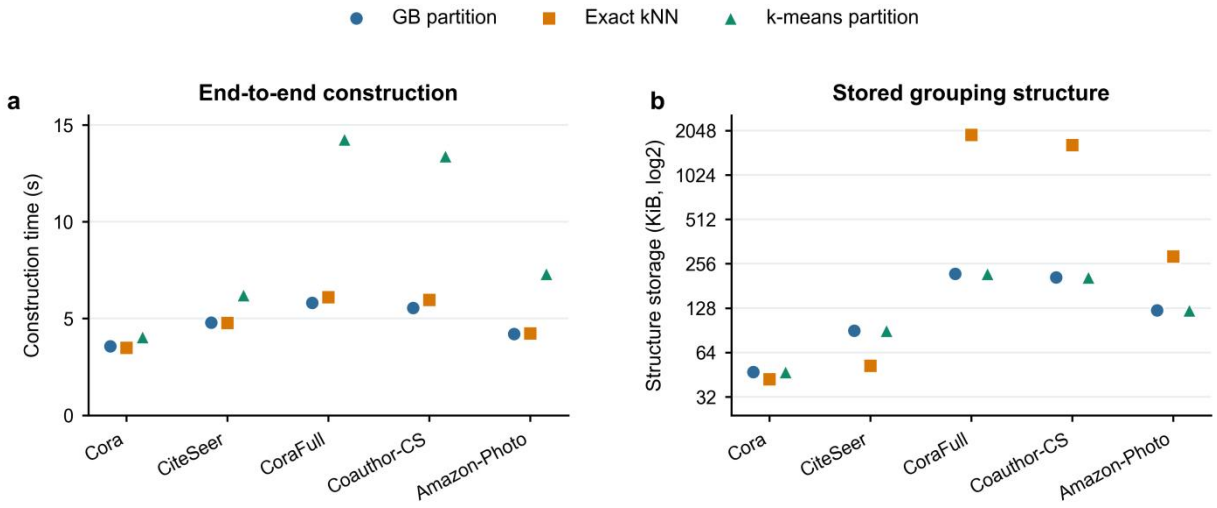


Fig. 5. Construction cost of granular ball, exact-kNN, and k-means grouping. (a) End-to-end construction time. (b) Stored grouping structure. Both are seed-0 measurements.

4.5 Discussion

GB-LCSS supplements an individual annotation with labels from related training nodes. Label-free grouping keeps corrupted annotations from shaping the groups, while excluding the current node prevents its label from reinforcing the consensus used to adjust it. Proposition 1 quantifies the reliability of the local vote when the true class is more frequent than any competing class among the voting nodes and labels are corrupted independently by symmetric noise. The random-grouping and smoothing controls (Table 5) support the value of selecting informative local evidence. The target-error decomposition in Appendix I.2 further shows how the targets improved: the reduction in error on corrupted labels outweighed the increase on initially correct labels in the tested runs.

The usefulness of this evidence depends on the labels within each group. Mixed-class groups may provide little guidance, whereas an incorrect local majority can bias the target. When observed labels are already correct, incorporating regional consensus may also weaken otherwise accurate supervision. This interpretation is consistent with the clean-label control, where MLP+CE achieves higher accuracy on four of five datasets (Appendix D). The additional-dataset results reveal a related distinction: GB-LCSS exceeds random grouping on all six datasets but the strongest noisy-label baseline on only three (Appendix G). Preserving meaningful local relationships therefore helps target construction, but the resulting benefit is not sufficient to outperform other robust learning strategies in every setting.

These findings place GB-LCSS as a method for improving noisy-label supervision while retaining graph-free inference. Graph-MLP and GLNN share the same attribute-only deployment interface. GB-LCSS groups nodes according to a label-free graph representation and uses the observed training labels within each group to refine supervision. Its mixed ranking against Graph-MLP in Table 4 indicates that this approach and graph-based training objectives offer different benefits across datasets. The teacher extension also shows that local-consensus targets can complement graph-to-MLP distillation: replacing observed labels with these targets improves student accuracy in nine of ten settings. This extension adds GCN teacher training and distillation over all nodes, while retaining the same deployed MLP.

Granular balls organize this supervision into shared regions. Each node is assigned to one group, class statistics are reused within that group, and the prescribed budget bounds the number of regions. The sensitivity

results in Section 4.3.4 show why local grouping and voting coverage matter: the largest losses occurred when all nodes remained in one ball or substantially fewer nodes received consensus supervision. In Table 6, granular ball grouping achieves accuracy within one percentage point of both kNN and k-means on four of the five datasets; on CoraFull, kNN is higher by 3.11 points. Granular ball grouping also uses less storage than exact kNN on the three larger datasets and has the shortest construction time on three of the five datasets in the reported measurements (Fig. 5). Together, these results support granular balls as a practical choice for combining competitive accuracy with shared regional statistics and favorable grouping costs.

5 Conclusion

Learning from noisy labels remains a challenge for node classifiers that must operate without graph access at inference. GB-LCSS addresses this problem by constructing granular balls in a frozen, label-free graph representation and combining regional label consensus with each node's observed label. This separates region construction from noisy-label aggregation and provides soft targets for an MLP that uses only node attributes at inference.

Across five datasets and five noise settings, GB-LCSS achieves or ties for the highest five-seed mean accuracy in 24 of 25 settings among seven noisy-label baselines evaluated using the same deployed MLP architecture. Controlled comparisons support representation-based regional supervision and generally favor excluding the current node's vote when forming consensus. The resulting soft targets have lower Brier scores than the observed one-hot targets, and the graph-teacher extension improves mean student accuracy over direct MLP training in all ten evaluated settings.

The current evaluation uses synthetic label noise and clean validation labels in a fixed-graph transductive setting. Future work will examine real annotation errors, model selection without clean validation labels, and inductive prediction for newly arriving nodes.

Acknowledgement

This work is supported by YANGTZE RIVER DELTA SOCIAL ASSISTANCE PUBLIC SERVICE PLATFORM (B801065001FY050) and the National Natural Science Foundation of China (Nos. 62276136, 62206129, and 62375133).

References

[1] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, Proc. Int. Conf. Learn. Represent. (2017).

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, Proc. Int. Conf. Learn. Represent. (2018).

[3] Y. Hu, H. You, Z. Wang, Z. Wang, E. Zhou, Y. Gao, Graph-MLP: Node classification without message passing in graph, arXiv:2106.04051 (2021).

[4] S. Zhang, Y. Liu, Y. Sun, N. Shah, Graph-less neural networks: Teaching old MLPs new tricks via distillation, Proc. Int. Conf. Learn. Represent. (2022).

[5] Z. Wang, Z. Zhang, C. Zhang, Y. Ye, Training MLPs on graphs without supervision, Proc. ACM Int. Conf. Web Search Data Min. (2025) 697–706.

[6] L. Wu, H. Lin, Y. Huang, T. Fan, S.Z. Li, Extracting low-/high-frequency knowledge from graph neural networks and injecting it into MLPs: An effective GNN-to-MLP distillation framework, Proc. AAAI Conf. Artif. Intell. 37 (2023) 10351–10360.

[7] C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning (still) requires rethinking generalization, Commun. ACM 64 (2021) 107–115.

[8] Z. Wang, D. Sun, S. Zhou, H. Wang, J. Fan, L. Huang, J. Bu, NoisyGL: A comprehensive benchmark for graph neural networks under label noise, Adv. Neural Inf. Process. Syst. 37 (2024) 38142–38170.

[9] E. Dai, C. Aggarwal, S. Wang, NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs, Proc. ACM SIGKDD Conf. Knowl. Discov. Data Min. (2021) 227–236.

[10] S. Qian, H. Ying, R. Hu, J. Zhou, J. Chen, D.Z. Chen, J. Wu, Robust training of graph neural networks via noise governance, Proc. ACM Int. Conf. Web Search Data Min. (2023) 607–615.

[11] Y. Wu, J. Yao, X. Xia, J. Yu, R. Wang, B. Han, T. Liu, Mitigating label noise on graphs via topological sample selection, Proc. Int. Conf. Mach. Learn., PMLR 235 (2024) 53944–53972.

[12] A. Iscen, J. Valmadre, A. Arnab, C. Schmid, Learning with neighbor consistency for noisy labels, Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (2022) 4672–4681.

[13] D. Bahri, H. Jiang, M. Gupta, Deep k-NN for noisy labels, Proc. Int. Conf. Mach. Learn., PMLR 119 (2020) 540–550.

[14] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, Proc. ICML Workshop Graph Represent. Learn. Beyond (2020).

[15] S. Xia, Y. Liu, X. Ding, G. Wang, H. Yu, Y. Luo, Granular ball computing classifiers for efficient, scalable and robust learning, Inf. Sci. 483 (2019) 136–152.

[16] S. Xia, S. Zheng, G. Wang, X. Gao, B. Wang, Granular ball sampling for noisy label classification or imbalanced classification, IEEE Trans. Neural Netw. Learn. Syst. 34 (2023) 2144–2155.

[17] S. Xia, X. Dai, G. Wang, X. Gao, E. Giem, An efficient and adaptive granular-ball generation method in classification problem, IEEE Trans. Neural Netw. Learn. Syst. 35 (2024) 5319–5331.

[18] S. Xia, G. Wang, X. Gao, X. Lian, Granular-ball computing: An efficient, robust, and interpretable adaptive multi-granularity representation and computation method, arXiv:2304.11171 (2023).

[19] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I.W. Tsang, M. Sugiyama, Co-teaching: Robust training of deep neural networks with extremely noisy labels, Adv. Neural Inf. Process. Syst. 31 (2018) 8527–8537.

[20] H. Wei, L. Feng, X. Chen, B. An, Combating noisy labels by agreement: A joint training method with co-regularization, Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (2020) 13726–13735.

[21] J. Li, R. Socher, S.C.H. Hoi, DivideMix: Learning with noisy labels as semi-supervised learning, Proc. Int. Conf. Learn. Represent. (2020).

[22] T. Kim, J. Ko, S. Cho, J. Choi, S.-Y. Yun, FINE samples for learning with noisy labels, Adv. Neural Inf. Process. Syst. 34 (2021) 24137–24149.

[23] W. Pan, W. Wei, F. Zhu, Y. Deng, Enhanced sample selection with confidence tracking: Identifying correctly labeled yet hard-to-learn samples in noisy data, Proc. AAAI Conf. Artif. Intell. 39 (2025) 19795–19803.

[24] Y. Wang, Y. Fu, X. Sun, Knockoffs-SPR: Clean sample selection in learning with noisy labels, IEEE Trans. Pattern Anal. Mach. Intell. 46 (2024) 3242–3256.

[25] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, J. Bailey, Symmetric cross entropy for robust learning with noisy labels, Proc. IEEE/CVF Int. Conf. Comput. Vis. (2019) 322–330.

[26] S. Liu, J. Niles-Weed, N. Razavian, C. Fernandez-Granda, Early-learning regularization prevents memorization of noisy labels, Adv. Neural Inf. Process. Syst. 33 (2020) 20331–20342.

[27] X. Ye, X. Li, S. Dai, T. Liu, Y. Sun, W. Tong, Active negative loss functions for learning with noisy labels, Adv. Neural Inf. Process. Syst. 36 (2023) 6917–6940.

[28] H. Wei, H. Zhuang, R. Xie, L. Feng, G. Niu, B. An, Y. Li, Mitigating memorization of noisy labels by clipping the model prediction, Proc. Int. Conf. Mach. Learn., PMLR 202 (2023) 36868–36886.

[29] Y. Li, J. Yin, L. Chen, Unified robust training for graph neural networks against label noise, Proc. Pacific-Asia Conf. Knowl. Discov. Data Min., Lect. Notes Comput. Sci. 12712 (2021) 528–540.

[30] Z. Wang, Y. Bei, S. Zhou, Z. Zhou, J. Fan, H. Xue, H. Wang, J. Bu, Learning from graph: Mitigating label noise on graph through topological feature reconstruction, Proc. ACM Int. Conf. Inf. Knowl. Manag. (2025) 3261–3270.

[31] K. Li, J. Sun, J. Lou, Z. Feng, H. Zhou, C. Wu, G. Xue, W. Zhao, J. Li, Leveraging peer-informed label consistency for robust graph neural networks with noisy labels, Proc. Int. Joint Conf. Artif. Intell. (2025) 5598–5606.

[32] K. Li, J. Lou, J. Sun, H. Zeng, W. Li, C. Wu, Y. Luo, W. Zhao, S. Du, J. Li, GD^2: Robust graph learning under label noise via dual-view prediction discrepancy, Adv. Neural Inf. Process. Syst. 38 (2025) 133248–133272.

[33] S. Li, B. Zhang, J. Song, G. Ruan, C. Wang, J. Xie, Graph neural networks with coarse- and fine-grained division for mitigating label noise and sparsity, Neural Netw. 187 (2025) 107338.

[34] K. Ding, X. Ma, Y. Liu, S. Pan, Divide and denoise: Empowering simple models for robust semi-supervised node classification against label noise, Proc. ACM SIGKDD Conf. Knowl. Discov. Data Min. (2024) 574–584.

[35] Y. Zhu, L. Feng, Z. Deng, Y. Chen, R. Amor, M. Witbrock, Robust node classification on graph data with graph and label noise, Proc. AAAI Conf. Artif. Intell. 38 (2024) 17220–17227.

[36] Q. Li, X. Li, D. Li, J. Wang, Prototype-guided supervision for graph learning with noisy and sparse labels, Proc. AAAI Conf. Artif. Intell. 40 (2026) 23105–23113.

[37] Y. Liu, Z. Wu, Z. Lu, C. Nie, G. Wen, Y. Zhu, X. Zhu, Noisy node classification by bi-level optimization based multi-teacher distillation, Proc. AAAI Conf. Artif. Intell. 39 (2025) 19033–19040.

[38] B. Hu, G. Wen, Y. Huang, Y. Liu, X. Zhu, Structure-aware multi-teacher distillation for noise node classification, Inf. Process. Manag. 63 (2026) 104877.

[39] M. Zhou, X. Han, W. Wei, G. Yan, Graph topology adaptive judgment against node label noise, Knowl.-Based Syst. 328 (2025) 114162.

[40] Z. Liu, H. Yu, X. Luo, Mitigating label noise in graph learning with information bottleneck-guided aggregation and adversarial consistency, Pattern Recognit. 180 (2026) 114271.

[41] S. Li, B. Zhang, M. Cao, J. Song, X. Cheng, C. Wang, HALF: A homophily-aware loss fusion for robust learning under label noise in heterophilic graphs, Inf. Sci. 754 (2026) 123714.

[42] L. Wu, J. Xia, H. Lin, Z. Gao, Z. Liu, G. Zhao, S.Z. Li, Teaching yourself: Graph self-distillation on neighborhood for node classification, arXiv:2210.02097 (2022).

[43] M. Kohn, M. Hoffmann, A. Scherp, Edge-splitting MLP: Node classification on homophilic and heterophilic graphs without message passing, Proc. Third Learn. Graphs Conf., PMLR 269 (2025) 11:1–11:21.

[44] Y. Tian, C. Zhang, Z. Guo, X. Zhang, N.V. Chawla, Learning MLPs on graphs: A unified view of effectiveness, robustness, and efficiency, Proc. Int. Conf. Learn. Represent. (2023).

[45] J. Chen, M. Bai, S. Chen, J. Gao, J. Zhang, J. Pu, SA-MLP: Distilling graph knowledge from GNNs into structure-aware MLP, Trans. Mach. Learn. Res. (2024).

[46] L. Wu, H. Lin, Y. Huang, S.Z. Li, Quantifying the knowledge in GNNs for reliable distillation into MLPs, Proc. Int. Conf. Mach. Learn., PMLR 202 (2023) 37571–37581.

[47] L. Yang, Y. Tian, M. Xu, Z. Liu, S. Hong, W. Qu, W. Zhang, B. Cui, M. Zhang, J. Leskovec, VQGraph: Rethinking graph representation space for bridging GNNs and MLPs, Proc. Int. Conf. Learn. Represent. (2024).

[48] T. Wu, Z. Zhao, J. Wang, X. Bai, L. Wang, N. Wong, Y. Yang, Edge-free but structure-aware: Prototype-guided knowledge distillation from GNNs to MLPs, Proc. Int. Conf. Comput. Linguist. (2025) 5656–5667.

[49] Z. Zhou, Z. Ding, J. Shi, L. Qing, S. Shen, TINED: GNNs-to-MLPs by teacher injection and Dirichlet energy distillation, Proc. Int. Conf. Mach. Learn., PMLR 267 (2025) 78616–78632.

[50] W. Lu, Z. Guan, W. Zhao, Y. Yang, AdaGMLP: AdaBoosting GNN-to-MLP knowledge distillation, Proc. ACM SIGKDD Conf. Knowl. Discov. Data Min. (2024) 2060–2071.

[51] W. Lu, Z. Guan, W. Zhao, Y. Yang, Y. Sun, Z. Liang, Y. Zhan, D. Tao, ProGMLP: A progressive framework for GNN-to-MLP knowledge distillation with efficient trade-offs, Proc. AAAI Conf. Artif. Intell. 40 (2026) 24088–24096.

[52] H. Pal, S. Malik, R. Patel, A. Malhotra, SALE-MLP: Structure aware latent embeddings for GNN to graph-free MLP distillation, Proc. Int. Joint Conf. Artif. Intell. (2025) 6003–6011.

[53] Z. Yu, M. Lin, W. Li, Z. Yin, S. Xiao, S. Lu, Demystifying GNN-to-MLP knowledge transfer: Theoretical grounding and dual-stream distillation method, Proc. AAAI Conf. Artif. Intell. 40 (2026) 28005–28013.

[54] J. Heo, M. Park, D. Kim, Posterior label smoothing for node classification, Proc. AAAI Conf. Artif. Intell. 40 (2026) 21708–21716.

[55] K. Zhou, S.-H. Choi, Z. Liu, N. Liu, F. Yang, R. Chen, L. Li, X. Hu, Adaptive label smoothing to regularize large-scale graph training, Proc. SIAM Int. Conf. Data Min. (2023) 55–63.

[56] Y. Wang, Y. Cai, Y. Liang, W. Wang, H. Ding, M. Chen, J. Tang, B. Hooi, Structure-aware label smoothing for graph neural networks, arXiv:2112.00499 (2021).

[57] Y. Xie, Z. Xu, J. Zhang, Z. Wang, S. Ji, Self-supervised learning of graph neural networks: A unified review, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2023) 2412–2429.

[58] J. Xie, X. Xiang, S. Xia, L. Jiang, G. Wang, X. Gao, MGNR: A multi-granularity neighbor relationship and its application in KNN classification and clustering methods, IEEE Trans. Pattern Anal. Mach. Intell. 46 (2024) 7956–7972.

[59] F. Dunkin, X. Li, H. Li, G. Wu, C. Hu, S.S. Ge, MgCNL: A sample separation approach via multi-granularity balls for fault diagnosis with the interference of noisy labels, IEEE Trans. Autom. Sci. Eng. 22 (2025) 7748–7761.

[60] F. Dunkin, X. Li, C. Hu, G. Wu, H. Li, X. Lu, Z. Zhang, Like draws to like: A multi-granularity ball-intra fusion approach for fault diagnosis models to resists misleading by noisy labels, Adv. Eng. Inform. 60 (2024) 102425.

[61] X. Peng, P. Wang, S. Xia, C. Wang, W. Chen, VPGB: A granular-ball based model for attribute reduction and classification with label noise, Inf. Sci. 611 (2022) 504–521.

[62] J. Zhang, X. Sun, B. Huang, T. Wang, H. Li, X. Jia, GEAF: A label noise filtering method based on adaptive partitioning of granular ellipsoids, Inf. Sci. 742 (2026) 123324.

[63] Z. Wang, T. Zhang, S. Xia, L. Lin, G. Wang, GBRAIN: Combating textual label noise by granular-ball based robust training, Proc. ACM Int. Conf. Multimed. Retr. (2024) 357–365.

[64] J. Yang, Q. Liu, X. Cao, F. Li, X. Yang, G-GBC: A Gaussian mixture-based granular ball computing method for robust CNN classification under noisy labels, Knowl.-Based Syst. 341 (2026) 115754.

[65] Y. Li, X. Ouyang, J. Zhang, C. Pan, L. Ren, X. Yang, Beneficial noise learning for open intent classification via granular-ball representation, Pattern Recognit. 177 (2026) 113283.

[66] S. Xia, X. Ma, Z. Liu, C. Liu, S. Zhao, G. Wang, Graph coarsening via supervised granular-ball for scalable graph neural network training, Proc. AAAI Conf. Artif. Intell. 39 (2025) 12872–12880.

[67] S. Xia, G. Wang, G. Xu, S. Zhao, G. Wang, GBGC: Efficient and adaptive graph coarsening via granular-ball computing, Proc. Int. Joint Conf. Artif. Intell. (2025) 3489–3497.

[68] J. Ni, S. Xia, Y. Guan, G. Xu, L. Chen, Y. Liu, Y. Wang, Multi-granularity graph refinement via granular ball for graph classification, Neurocomputing 686 (2026) 133713.

[69] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, T. Eliassi-Rad, Collective classification in network data, AI Mag. 29 (2008) 93–106.

[70] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, Proc. NeurIPS Workshop Relational Represent. Learn. (2018).

[71] D. Arthur, S. Vassilvitskii, k-means++: The advantages of careful seeding, Proc. Eighteenth Annu. ACM-SIAM Symp. Discrete Algorithms (2007) 1027–1035.

[72] S. Holm, A simple sequentially rejective multiple test procedure, Scand. J. Stat. 6 (1979) 65–70.

[73] G.W. Brier, Verification of forecasts expressed in terms of probability, Mon. Weather Rev. 78 (1950) 1–3.

[74] L. Hubert, P. Arabie, Comparing partitions, J. Classif. 2 (1985) 193–218.